.NET Core Interview Questions and Answers in 2025

1. What is .NET Core, and how is it different from the .NET Framework?

.NET Core is a cross-platform, high-performance framework for building modern cloud-based, internet-connected applications. Unlike the .NET Framework, which runs only on Windows, .NET Core supports Windows, macOS, and Linux, making it more versatile for cross-platform development.

2. How do you install .NET Core SDK?

You can download the .NET Core SDK from the official .NET website. Follow the installation guide for your specific operating system (Windows, macOS, or Linux).

3. What is the purpose of the dotnet new command?

The dotnet new command creates a new .NET Core project with a specific template (e.g., console app, web app, etc.). For example, dotnet new console creates a new console application.

4. How does .NET Core support cross-platform development?

.NET Core is designed to be platform-independent. It uses a runtime (CoreCLR) that can run on different operating systems. The same .NET Core application can run on Windows, macOS, and Linux without modification.

5. What are the main benefits of using .NET Core for application development?

1. Cross-platform support (Windows, macOS, Linux)
2. High performance and scalability

3. Modern development (support for cloud, microservices)
4. Open-source and community-driven
5. Unified development (build web, desktop, mobile, and cloud apps)

6. What is the role of the Common Language Runtime (CLR) in .NET Core?

The CLR is the runtime environment in .NET Core that manages the execution of .NET applications, providing services like memory management, exception handling, and garbage collection.

7. Explain the structure of a .NET Core project.

A typical .NET Core project contains files like Program.cs (entry point), Startup.cs (configuration), and appsettings.json (configuration settings). It also includes directories like Controllers, Views, wwwroot, and dependencies listed in the csproj file.

8. What is the difference between dotnet run and dotnet build commands?

1. dotnet build compiles the application without running it.
2. dotnet run compiles and runs the application.

9. How do you create a new ASP.NET Core MVC project from scratch?

Run the command dotnet new mvc -n ProjectName to create a new ASP.NET Core MVC project. Then, navigate to the project directory and run dotnet run to launch the project.

10. What is Kestrel, and why is it used in .NET Core?

Kestrel is a cross-platform web server for ASP.NET Core applications. It is used to serve HTTP requests and can run behind other web servers like IIS or Nginx, or as a standalone server.

11. How does .NET Core support dependency injection?

.NET Core has built-in support for dependency injection (DI) through the IServiceCollection and IServiceProvider interfaces. You can register services in the Startup.cs file using methods like AddSingleton, AddScoped, and AddTransient.

12. Explain what middleware is in ASP.NET Core.

Middleware is software that handles requests and responses in an ASP.NET Core application. Middleware components are executed in a pipeline, where each component can either pass the request to the next component or terminate the pipeline.

13. How does routing work in ASP.NET Core MVC?

Routing in ASP.NET Core MVC maps incoming requests to controller actions. Routes are defined in the Startup.cs file, typically using the UseEndpoints or MapControllerRoute methods.

14. What is an API controller, and how does it differ from an MVC controller in .NET Core?

API controllers are specifically used for building Web APIs. They return data (e.g., JSON) rather than views. API controllers use the [ApiController] attribute to enable features like automatic model validation.

15. What is appsettings.json, and how do you configure settings in it?

appsettings.json is a configuration file used in .NET Core applications to store settings like database connection strings, environment settings, and more. You can access these settings through the IConfiguration interface in .NET Core.

16. What is Entity Framework Core, and what is its primary use?

Entity Framework Core (EF Core) is an Object-Relational Mapping (ORM) framework that allows developers to interact with databases using .NET objects, eliminating the need for most SQL queries.

17. How do you connect a .NET Core application to a SQL Server database?

Add a connection string to appsettings.json, then configure it in Startup.cs using the AddDbContext method. You can then inject the DbContext into your application to interact with the database.

18. How does the IConfiguration interface work in .NET Core?

The IConfiguration interface provides access to configuration settings, allowing you to read key-value pairs from files like appsettings.json, environment variables, or command-line arguments.

19. What are NuGet packages, and how are they used in .NET Core projects?

NuGet is a package manager for .NET. NuGet packages are libraries that you can add to your project to use third-party or shared code. You can install them via the dotnet add package command.

20. Explain how you handle errors using try-catch in .NET Core.

The try-catch block is used to handle exceptions. You place the code that might throw an exception inside the try block and handle the exception in the catch block.

21. What is the role of the Program.cs file in a .NET Core project?

Program.cs is the entry point of a .NET Core application. It contains the Main method, which starts the application by calling CreateHostBuilder and setting up the host.

22. What is Startup.cs, and what are its responsibilities?

Startup.cs configures the services and the request pipeline for the application. It defines the ConfigureServices method for DI configuration and Configure method for request processing pipeline configuration.

23. How do you enable HTTPS in a .NET Core application?

To enable HTTPS, configure your project to use HTTPS in launchSettings.json. Also, add middleware like UseHttpsRedirection in the Startup.cs file.

24. How do you implement basic logging in .NET Core?

.NET Core provides built-in logging support through the ILogger interface. You can inject ILogger into controllers or services and log information using methods like LogInformation, LogWarning, and LogError.

25. What is model binding in ASP.NET Core MVC?

Model binding is the process of mapping incoming request data (such as form data or query parameters) to action method parameters.

26. Explain what ViewData, ViewBag, and TempData are in ASP.NET Core.

1. ViewData: A dictionary for passing data from controller to view.
2. ViewBag: A dynamic wrapper around ViewData for easier access to data.
3. TempData: Used to store data temporarily between two requests, typically after a redirect.

27. How do you implement validation in ASP.NET Core using data annotations?

Data annotations are attributes applied to model properties to enforce validation. For example, [Required], [Range], and [StringLength] are used for validation. These attributes are checked automatically by the model binder.

28. What is a RESTful API, and how do you build one using ASP.NET Core?

A RESTful API follows REST principles like stateless communication, resource representation, and HTTP methods. You can create a RESTful API in ASP.NET Core by creating controllers that return data (usually JSON) and use HTTP methods like GET, POST, PUT, and DELETE.

29. How do you consume a Web API in a .NET Core application?

You can use HttpClient to consume a Web API. Create an instance of HttpClient, send a request using GetAsync or PostAsync, and process the response.

30. How do you manage cookies in an ASP.NET Core application?

You can manage cookies using the HttpContext.Response.Cookies.Append method to set cookies and HttpContext.Request.Cookies to read them.

31. What are filters in ASP.NET Core?

Filters allow you to run code before or after an action method executes. Examples include authorization filters, resource filters, action filters, and exception filters.

32. How do you handle user authentication in an ASP.NET Core application?

ASP.NET Core supports authentication through Identity, JWT, or OAuth. You configure authentication in Startup.cs and use middleware to handle login, registration, and token validation.

33. What is Razor, and how is it used in ASP.NET Core MVC?

Razor is a templating engine used to generate dynamic HTML in MVC views. It allows you to embed C# code into HTML using the @ symbol.

34. How do you serve static files in a .NET Core application?

Use the Use Static Files middleware in Startup. cs to serve static files from the www root folder.

35. What is the View Component in ASP.NET Core MVC?

A View Component is a reusable component that renders part of a page. It is similar to partial views but allows for more complex logic.

36. How do you return JSON data from a controller in ASP.NET Core?

Return a JSON result from an action method using return J son(object) or return Ok(object) in an API controller.

37. How do you perform file uploads in ASP.NET Core?

Use the I Form File interface to handle file uploads. In the controller, read the uploaded file, process it, and save it to a specified location.

38. What are Tag Helpers in ASP.NET Core?

Tag Helpers are server-side components that generate HTML and provide a way to interact with Razor views. Examples include asp-for, asp-controller, and asp-action.

39. How do you publish a .NET Core application?

Use the dotnet publish command to compile and package the application for deployment. You can also specify the target runtime (e.g., Windows, Linux) during the publishing process.

40. What are global exception handlers, and how do you implement them in .NET Core?

Global exception handling can be implemented using middleware. You create custom middleware that catches exceptions and handles them centrally, ensuring that all unhandled exceptions are captured.

41. How do you implement Dependency Injection (DI) in .NET Core?

Dependency Injection (DI) is a design pattern used to implement IoC (Inversion of Control), allowing you to manage the dependencies of classes. In .NET Core, DI is a first-class citizen, and the framework provides built-in support for it.

To implement DI in .NET Core:

Register services: You define the service in the Startup.cs file, inside the ConfigureServices method. You can register services with different lifetimes like Transient, Scoped, or Singleton.

42. Explain the difference between IActionResult and ActionResult in an ASP.NET Core controller.

Both IActionResult and ActionResult are used as return types for ASP.NET Core controller actions, but there are some key differences:

IActionResult:

1. IActionResult is an **interface** that represents a wide variety of possible return types for controller actions.
2. It allows for greater flexibility as it can represent any type that implements IActionResult, such as ViewResult, JsonResult, RedirectResult, ContentResult, etc.

43. What is model validation, and how do you handle it in ASP.NET Core MVC?

Model validation is the process of ensuring that incoming data from HTTP requests conforms to the rules defined in your data models. ASP.NET Core provides a built-in mechanism to validate models using **data annotations**.

Defining validation rules: Use data annotations to apply validation rules to your model properties:

44. What is Middleware in ASP.NET Core? How does it work?

Middleware in ASP.NET Core is software that sits between the incoming HTTP request and the outgoing HTTP response. It is responsible for handling and processing HTTP requests, and each piece of middleware can either:

- **Process the request** and generate a response, or
- **Pass the request to the next middleware** in the pipeline.

Middleware components are executed in the order they are registered in the Startup.cs file, and each one has the option to terminate the request by generating its own response or delegate the request to the next middleware

45. What are Filters in ASP.NET Core MVC? What are the types of filters?

Filters in ASP.NET Core MVC are used to execute custom logic before or after certain stages in the request pipeline, such as before an action is executed or after a result is returned. Filters provide a way to implement cross-cutting concerns like logging, authorization, caching, and exception handling.There are several types of filters:

Authorization Filters:

1. Run before the action is executed to determine whether the user is authorized to execute the action.
2. Example: [Authorize] attribute is an authorization filter.

46. What are Tag Helpers in ASP.NET Core?

Tag Helpers in ASP.NET Core are used to extend the functionality of HTML elements in Razor views. They allow you to add server-side processing logic to HTML tags in a clean and readable way, making your Razor views more expressive and maintainable.Tag Helpers are processed on the server, and they help you build dynamic content while still working with familiar HTML syntax.

47. How does ASP.NET Core support Cross-Origin Resource Sharing (CORS)?

Cross-Origin Resource Sharing (CORS) is a security feature implemented by web browsers to prevent unauthorized access to resources from different origins (domains). ASP.NET Core provides built-in support for configuring CORS policies to allow or restrict cross-origin requests.

1. **Enable CORS in Startup.cs**: You need to configure CORS in the Startup.cs file by using the services.AddCors() method inside ConfigureServices and applying CORS policies in Configure().

48. What is View Model in ASP.NET Core MVC? How is it different from a Model?

In ASP.NET Core MVC, a **View Model** is a class that represents the data and logic required specifically for the view. It is designed to transfer data between the controller and the view, and is often a combination of multiple models or properties tailored for the view's needs. **Difference between View Model and Model:**

Model:

1. Represents the domain or business logic of the application.
2. Contains data and validation rules for entities like User, Product, etc.
3. Used to interact with the database through ORM frameworks like Entity Framework.

ViewModel:

1. A custom class designed for the view, containing only the data needed for display or user interaction.
2. Can include properties from multiple models, as well as additional logic or formatting specific to the view.
3. Not tied to the database.

49. What is the purpose of the Configure Services method in Startup. cs?

The **Configure Services** method in ASP.NET Core's Startup. cs file is used to configure the **Dependency Injection (DI) container**. It is where services are registered for use in the application. ASP.NET Core uses this method to register services such as MVC, custom services, middleware components, authentication services, database contexts, and more.

50. How do you implement Authentication and Authorization in ASP.NET Core?

Authentication and **Authorization** are essential aspects of securing ASP.NET Core applications.

1. **Authentication**:
    - Authentication is the process of verifying the user's identity (e.g., login credentials).
    - ASP.NET Core supports various authentication mechanisms such as **cookies**, **JWT tokens**, **OAuth**, and **OpenID Connect**.
2. **Authorization**:
    - Authorization determines what actions a user can perform after their identity has been authenticated.
    - ASP.NET Core uses **roles** or **claims** for implementing authorization.

51. What is Middleware in ASP.NET Core, and how does it work?

Middleware in ASP.NET Core is software that is assembled into the application's **request pipeline** to handle requests and responses. Each middleware component can either process an incoming request and pass it to the next middleware in the pipeline or terminate the request by generating a response.

Middleware is executed in the order it is registered in the Configure method, which makes the order of middleware components critical.

Common ASP.NET Core middleware components:

- **UseRouting**: Sets up endpoint routing.
- **UseAuthentication**: Handles authentication.
- **UseAuthorization**: Handles authorization based on user roles or claims.
- **UseStaticFiles**: Serves static files like CSS, JavaScript, and images.
- **UseExceptionHandler**: Handles exceptions globally.

Creating Custom Middleware: You can create custom middleware by implementing a class that takes an HttpContext object, processes the request, and either generates a response or passes the request to the next middleware component.

53. What is ViewComponent in ASP.NET Core, and how is it different from a PartialView?

A **ViewComponent** in ASP.NET Core is a reusable component that encapsulates rendering logic, similar to a controller action but with the purpose of rendering a specific part of a view. ViewComponents are ideal for building complex UI components that may include their own logic and models, independent of the main page or view model.

Key features of ViewComponent:

1. It is **reusable** across multiple views.
2. It has its own **logic** and **view**.
3. It doesn't participate in the MVC action pipeline but is invoked directly from a view.
4. It can return **HTML**, **JSON**, or other formats.

Differences between ViewComponent and PartialView:

- **ViewComponent**: Contains both rendering logic (C# code) and a view (UI part). It has its own model and can perform complex logic before rendering the view.
- **PartialView**: Is purely a view (UI component) without any logic. It shares the model of the parent view.

54. How does session management work in ASP.NET Core?

In ASP.NET Core, **session management** is used to store **temporary data** specific to a user across multiple HTTP requests. Unlike cookies, session data is stored server-side, and a **session ID** is passed to the client via a cookie.

How session management works:

1. Session data is stored in the server's memory, a distributed cache, or a database.
2. The session ID is stored in a cookie on the client.
3. Each time the user makes a request, the session ID is sent with the request, and the server retrieves the corresponding session data.

55. What is UseEndpoints() in ASP.NET Core, and how does it differ from UseMvc()?

UseEndpoints() and UseMvc() are methods in ASP.NET Core used to configure request handling, but they differ in how they manage routing and endpoints.

UseMvc():

1. Part of the **older routing system** in ASP.NET Core 2.x.
2. Configures routing based on controller actions, with routes defined explicitly in Startup.cs or using attribute-based routing.
3. It was commonly used with **services.AddMvc()** in the ConfigureServices() method.